

Course:

OBJECT ORIENTED PROGRAMMING (JAVA)

Assignment Type:

THEORY # 01

Topic:

SMART CAMPUS ALLOCATION SYSTEM

Submitted by:	
Name:	Sara Asghar
Section:	SP26-BAI-A
Roll Number:	044
Submitted to:	
Name:	Muhammad Shahid Bhatti
Course Number:	CSC241
Date of Submission:	22/09/26

Problem Scenario:

COMSATS Lahore plans to introduce a **Smart Campus Learning Space Allocation System** for classrooms, laboratories, seminar rooms, and project spaces. Students, faculty members, administrators, security staff, maintenance staff, and disability support staff will use or depend on the system. The university has not prescribed one design. You must analyze the problem and propose an object oriented conceptual model. **(No Java implementation is required)**

Task 01 — Stakeholder and Requirement Analysis

Stakeholders:

1. Needs:

- **Student:** demand fair access to study rooms and equipment without lower-priority requests being cancelled without notice.
- **Faculty:** require priority booking for scheduled classes, academic events with guaranteed equipment functionality.
- **Administration Staff:** require real-time reporting to optimize space utilization.
- **Maintenance Staff:** require authority to put rooms under lockdown(**maintenance**) for safety purposes.

- **Disability Support Officers:** require the ability to verify accessibility needs and manage room allocation flags along with preserving user privacy.

2. Requirement Conflicts:

Requirement conflicts	Conflict	Decision/Assumption
User Privacy vs Administrative Verification:	Support officers must verify eligibility, but public schedules must not leak medical status.	System uses a masked flag (hasClearance); schedules only show BOOKED.
Faculty priority vs Student Booking Stability:	Faculty needs priority, but sudden student cancellations cause unfairness.	Faculty takeover requires 24-hour notice; bookings within 24 hours are locked.
Accessibility Priority vs. General Fair Access:	Holding accessible spaces risks low utilization, but open booking locks out disabled users.	Accessible rooms hold priority time slot for 12 hours then are open to all users.

Facts vs. Assumptions:

- **Facts:** Campus has multiple room types and roles; accessible rooms must be available for disability needs; **personal medical data** must remain private.
- **Assumptions:** Bookings are made in 30 minute slots, eligibility flag is a mask, and that the accessible rooms which have not been booked auto release 12 hours prior to start time.

Task 02 — Object Oriented Conceptual Model

Classes, responsibilities and relevant attributes

Class Name	Responsibility	Attributes	Operations
User	represents any person interacting with the system, manages basic profile information and identity authentication.	userID: String name: String	getRoles(): List<Role> hasClearance(feature: AccessibilityFeature): boolean
Role	encapsulates permission and booking priority levels associated with a specific user role (e.g., Student, Faculty, Disability Support Officer, Maintenance Staff).	priorityLevel: int roleType: RoleType (constant: STUDENT, FACULTY, MAINTENANCE, DISABILITY_OFFICE R, etc.)	canOverrideBooking(existingBooking : Reservation): boolean getBookingLimit(): int
Room	represents a space (classroom, lab, seminar room) on campus and tracks its physical characteristics and availability status.	roomID: String capacity: int isLockedDown: boolean	isAvailable(timeslot: TimeSlot): boolean lockdownRoom()

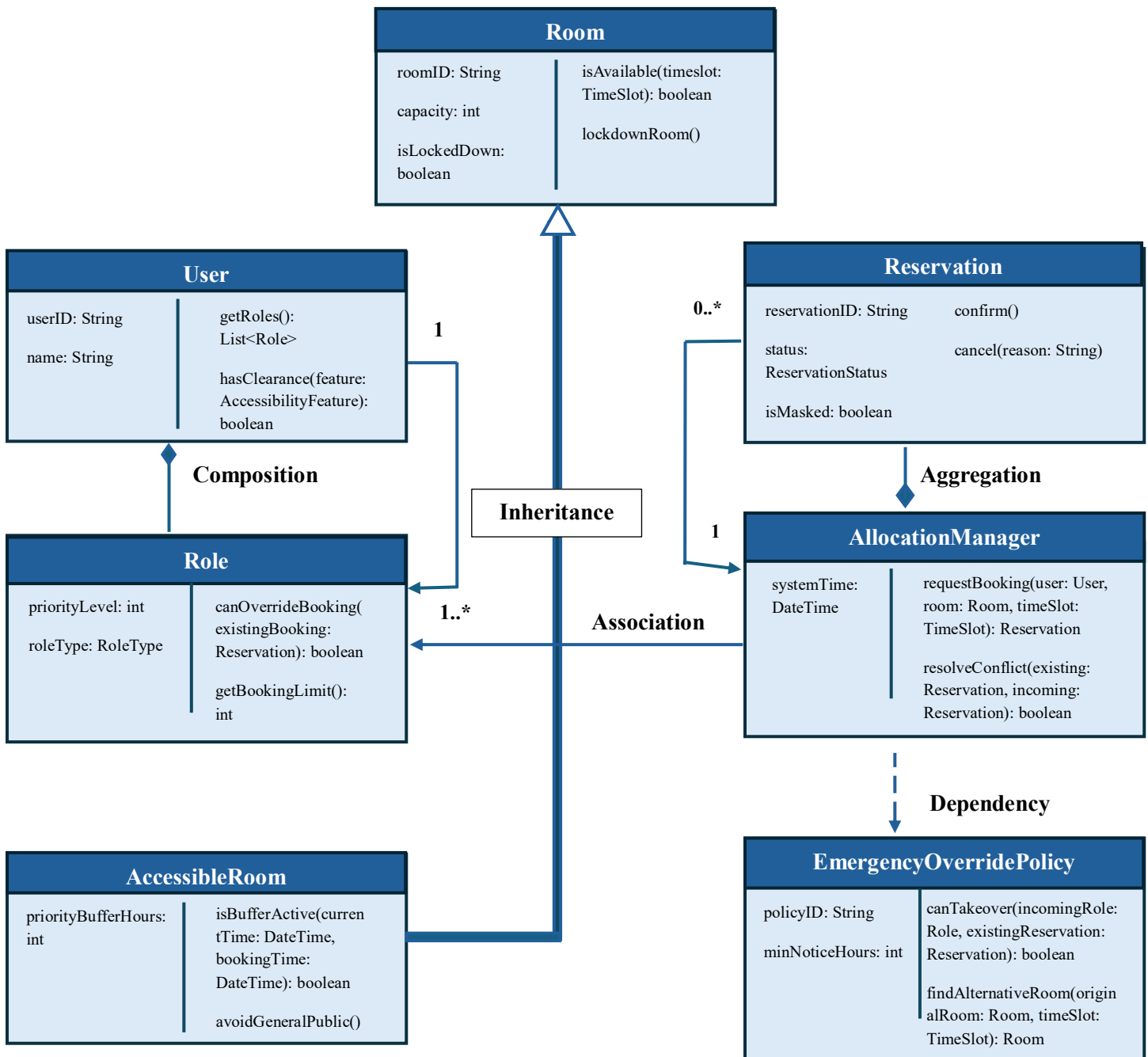
Accessible Room	a specialized room with disability support features; enforces the priority buffer mechanism to preserve accessibility without violating privacy.	priorityBufferHours: int (e.g., 12 hours before start time)	isBufferActive(currentTime: DateTime, bookingTime: DateTime): boolean avoidGeneralPublic()
Reservation	tracks the lifecycle, duration, and state of a room allocation request.	reservationID: String status: ReservationStatus (Enum/constant: PENDING, CONFIRMED, CANCELLED, COMPLETED, LOCKED) isMasked: boolean	confirm() cancel(reason: String)
Allocation Manager	controller responsible for processing reservation requests, validating priority rules, applying conflict policies.	systemTime: DateTime	requestBooking(user: User, room: Room, timeSlot: TimeSlot): Reservation resolveConflict(existing: Reservation, incoming: Reservation): boolean
Emergency OverridePolicy	Encapsulates business rules for handling urgent building closures, maintenance lockdowns, or faculty takeovers.	policyID: String minNoticeHours: int	canTakeover(incomingRole: Role, existingReservation: Reservation): boolean findAlternativeRoom(originalRoom: Room, timeSlot: TimeSlot): Room

Responsibility Separation and Avoiding a God Class

In order to avoid a single “god class” that controls the entire system, the tasks are distributed to different classes to apply the **Single Responsibility Principle (SRP)**:

- **User Identity & Authorization (User & Role):** the User class manages user profile information whereas the Role class manages priority and booking limits, as opposed to permission rules for profile management.
- **State Logic (Reservation & EmergencyOverridePolicy):** Reservation is able to track its own state (e.g. CONFIRMED, CANCELLED) whereas the EmergencyOverridePolicy deals with the rules for urgent cancellations and takeovers.
- **Space Management (Room, AccessibleRoom, & Equipment):** Room tracks availability and lockdown status for regular space bookings. AccessibleRoom includes priority buffers and privacy flags for bookings.

UML Class Diagram:



Task 03 — Abstraction, Encapsulation and Invariants

1. Abstraction

- User:** Real-world abstraction of a campus member, prevents exposure of personal details retaining only properties essential for system authorization: **name**, **ID**, and **clearance flags**.

- **Room:** Real-world abstraction of a room, masks structural/aesthetic details retaining only operations: lockdown, capacity and availability status.
- **Reservation:** Real-world abstraction of a room allocation agreement, masks personal information taking only structural data: status and privacy masking flags.

2. Encapsulation

- **User:**
 - **Masked Information:** Disability support clearance flags.
 - **Operations:** **getRoles()** to view assigned roles safely, and **hasClearance()** to verify accessibility without exposing medical details.
- **Room:**
 - **Masked Information:** lockdown flag (**isLockedDown**).
 - **Operations:** **lockdownRoom()** to restrict access during safety events, and **isAvailable()** for scheduling.
- **Reservation:**
 - **Masked Information:** Status constant/Enum (**status**) and public masking flag (**isMasked**).
 - **Operations:** **confirm()** to confirm state transitions and **cancel(reason: String)** to enforce cancellation.

3. Invariants and State Integrity

1. **No Double Bookings:** You cannot have two active/reserved and confirmed reservations (**status = CONFIRMED**) for the same time slot.
2. **Capacity Consideration:** A reservation for a Room cannot be placed if the number of participants requested is greater than the maximum size of the Room for the requested number of participants.
3. **Lockdown for Safety:** When the **Room.isLockedDown** flag is true then no new reservations should be created and all current reservations for that room should be changed to **CANCELLED** status.
4. **Protection of Privacy:** Masking the medical background of a user by preventing its exposure to the general public.

Prevention of Invalid State by Encapsulation: Making fields like **isLockedDown** and **status** private forces state changes to pass through methods like **confirm()** or **lockdownRoom()**, these methods confirm preconditions before **changing variables** and **preventing double-booking**.

Task 04 — Alternative Designs and Decision

Comparison:

Alternatives:

For managing users with multiple changing roles:

- **Inheritance(Class):** Roles are subclassed directly from the User
Example: FacultyUser, StudentUser, TAUser
- **Composition:** A concrete User class holds a reference to a collection of distinct Role objects (List<Role>).

Comparison of Alternatives:

Criteria	Inheritance	Composition
Extensibility	Low Adding a new extension/role requires creating new subclasses in order to handle overlapping responsibilities.	High. Adding a new role simply requires creating a new instance or implementation of the Role class
Coupling/Multiple role	Poor A single user cannot inherit from StudentUser FacultyUser (needs to be duplicated across class boundaries)	Excellent. A single User instance can hold multiple Role objects in a List<Role> simultaneously.
Ease of changing permissions	Difficult Change of a student role during a semester (TA) requires an entire new user object	Continuous Permissions and roles can be updated dynamically at runtime by adding, removing, or swapping Role objects

Design Choice and its Justification:

Design Choice: Composition

Justification:

1. **Accessibility and Privacy:** Permission flags must be checked without revealing personal information (medical Status). Composition lets User allocate permission checks to its Role list clearly while preserving encapsulation.
2. **Multirole Support:** Supports users with overlapping identities (e.g., Student-TA) by providing multiple roles simultaneously to one account and **updating permissions at runtime**

Task 05 — Behaviour and Edge Cases

Reservation Lifecycle (State Transitions)

- **PENDING:** triggered when a user submits a request; system checks capacity and user permissions.
- **CONFIRMED:** triggered when AllocationManager approves the request, user is notified, and identity details are masked.
- **COMPLETED:** triggered when the reserved time slot successfully ends; space is auto-released.
- **CANCELLED:** triggered by user cancellation, maintenance lockdown (lockdownRoom()), override policy, or expiration of the 12-hour buffer.

- **RELOCATED:** triggered during emergency closures or lockdowns when EmergencyOverridePolicy successfully finds an alternative room.

Case Handling

Case 1: Two Simultaneous Requests for the Same Space

- **Resolution:** AllocationManager processes simultaneous requests and compares role priority levels.
- **Accessibility Rule:** requests for an AccessibleRoom are granted to **disabled users first**, and gives the losing request an alternative space.

Case 2: Maintenance Block After Confirmation

- **Resolution:** maintenance staff put room on lockdown, immediately causing CONFIRMED to become CANCELLED.
- **System Action:** AllocationManager calls EmergencyOverridePolicy.findAlternativeRoom() to relocate bookings to different rooms but if no room exists cancellation occurs.

Case 3: Emergency Cancellation or Relocation

- **Resolution:** urgent overrides through EmergencyOverridePolicy for verifying rules
- **System Action:** AccessibleRoom bookings are only changed if an equivalent space is available, without revealing private user data.

Decision Justification

- **AllocationManager:** acts as a controller and manages workflow and coordinates requests between users and rooms without cluttering.
- **Room / AccessibleRoom:** follows the information expert principle because it owns its **capacity and lockdown data**.
- **EmergencyOverridePolicy:** follows the **Single Responsibility Principle (SRP)** encapsulating administrative rules into a separate policy keeping booking logic clean.

Task 06 — Independent Exploration

Strategy Pattern:

The Strategy Pattern is a **behavioral object-oriented design** pattern that defines a family of algorithms, encapsulates each one into a separate class and makes them interchangeable at runtime [1], [2].

Instead of having **conditional branching logic** (multiple if-else or switch statements) inside a client object, the context delegates the job of execution to the implementation of a common strategy interface [2]. In my conceptual model, the Strategy pattern will directly improve room allocation conflict resolution inside the AllocationManager. Currently, the conflict checks and override logic is context dependent (for general bookings, emergency closures or priority buffers for AccessibleRoom instances).

By implementing an AllocationStrategy interface and having different firm classes implement the method, the AllocationManager can simply delegate the job **at runtime** and avoid changing transaction code for domain classes. This helps stick to the **Open/Closed Principle (OCP)** as administrators can add new scheduling policies (e.g. for exam periods) without corrupting/cmodifying existing classes.

References

- [1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
- [2] E. Freeman and E. Robson, Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software, O'Reilly Media, 2020.

AI Disclosure:

AI tools were used to verify/check my UML diagram and classes and were to be deemed as correct also to learn how to add citation/references in my word file document.